Operator	Meaning
==	Test for equal value.
===	Test for "matching" as defined by the type of the operand. Used most often to compare each of the items with the target in the when clause of a case statement.
<=>	General comparison operator. Returns -1, 0, or +1, depending on whether its receiver is less than, equal to, or greater than its argument.
<, <=, >=, >	Comparison operators for less than, less than or equal, greater than or equal, and greater than.
=~	Regular expression pattern match.
eql?	True if the receiver and argument have both the same type and equal values. 1 == 1.0 returns true, but 1.eql?(1.0) is false.
equal?	True if the receiver and argument have the same object ID.

Table 3—Common comparison operators

Both == and =~ have negated forms, != and !~. When evaluating the negated versions, Ruby first looks for methods called != or !~, calling them if found. If not, it will then invoke either == or =~, negating the result.

In the following example, Ruby calls the == method to perform both comparisons:

```
class T
  def ==(other)
    puts "Comparing self == #{other}"
    other == "value"
  end
end

t = T.new
p(t == "value")
p(t != "value")
```

produces:

```
Comparing self == value
true
Comparing self == value
false
```

If instead we explicitly define !=, Ruby calls it:

```
class T
  def ==(other)
    puts "Comparing self == #{other}"
    other == "value"
  end
  def !=(other)
    puts "Comparing self != #{other}"
    other != "value"
  end
end

t = T.new
p(t == "value")
p(t != "value")
```